🐧 Chatbot de Comandos Linux

Chatbot de escritorio en Python con interfaz gráfica (Tkinter) que responde preguntas sobre los comandos más comunes de Linux, con búsqueda inteligente tolerante a erratas.

> Enlace al repositorio

Contenidos:

- 🐧 Chatbot de Comandos Linux
  - 📖 Descripción del proyecto
    - ¿Qué es?
    - ¿Qué hace?
    - ¿Cómo funciona?
    - Estructura del repositorio
  - 🚀 Instrucciones de instalación y despliegue
    - Requisitos previos
    - 1. Clonar el repositorio
    - 2. Configurar la pantalla (solo Windows)
    - 3. Construir la imagen con `docker build`
    - 4. Levantar los servicios con `docker compose up`
    - 5. Acceder a la aplicación
    - 6. Exposición de puertos
  - 🐳 Explicación de los archivos Docker
    - `Dockerfile`
    - `docker-compose.yml`
  - 🔧 Posibles problemas y soluciones
  - ✨ Contribuciones y organización del proyecto
    - Flujo de trabajo con ramas
    - Pasos para contribuir
    - Convención de commits
    - Ampliar la base de conocimiento
  - 🖼️ Diagrama de arquitectura

📖 Descripción del proyecto

¿Qué es?

Una aplicación de escritorio tipo chatbot donde el usuario escribe el nombre de un comando Linux (`ls`, `grep`, `chmod`...) y recibe una descripción clara junto con un enlace a la documentación oficial.

¿Qué hace?

- Responde preguntas sobre 18 comandos Linux habituales.
- Búsqueda en dos pasos: primero coincidencia exacta de palabras clave y, si no hay resultado, búsqueda difusa con `difflib.SequenceMatcher` para detectar erratas.

- Indica el tipo de resultado con un código de color:
  - 🟢 `✓ coincidencia exacta`
  - 🟠 `~ coincidencia aproximada`
  - 🔴 `✗ sin resultado`
- Contador de preguntas realizadas en la sesión.
- Botón para limpiar el historial.

¿Cómo funciona?

```
Usuario escribe una pregunta
          │
          ▼
  ¿Coincide alguna palabra clave exacta?
     ├── SÍ ──► devuelve respuesta (🟢 verde)
     └── NO ──► búsqueda difusa con SequenceMatcher
                   ├── similitud ≥ 0.45 ──► respuesta aproximada (🟠 naranja)
                   └── similitud < 0.45 ──► sin resultado (🔴 rojo)
```

La base de conocimiento vive en `src/db.json`. Se puede ampliar añadiendo entradas sin tocar el código Python.

Estructura del repositorio

```
chatbot-linux/
├── src/
│   ├── app.py              # Código principal (Python + Tkinter)
│   └── db.json             # Base de conocimiento (comandos Linux)
├── docs/
│   └── img/                # Capturas de pantalla
├── Dockerfile
├── docker-compose.yml
├── .gitignore
└── README.md
```

🚀 Instrucciones de instalación y despliegue

Requisitos previos

- Docker Desktop instalado y en ejecución.
- VcXsrv instalado (solo Windows, para mostrar la ventana gráfica).
- Git instalado.

---

1. Clonar el repositorio

```
git clone https://github.com/little-shiny/chatbot-linux
cd chatbot-linux
```

---

2. Configurar la pantalla (solo Windows)

Paso 1 — Arrancar VcXsrv (XLaunch):

1. Display settings → **Multiple windows** → Next
2. Client startup → **Start no client** → Next
3. Extra settings → marca **Disable access control** → Finish

Paso 2 — Obtener tu IP local:

```
ipconfig
# Busca "Dirección IPv4", por ejemplo: 192.168.1.50
```

Paso 3 — Editar `docker-compose.yml` con tu IP:

```
environment:
  - DISPLAY=192.168.1.50:0.0   # <- pon aquí tu IP
```

En Linux no hace falta este paso, pero sí ejecutar:

```
xhost +local:docker
```

---

3. Construir la imagen con `docker build`

```
docker build -t chatbot-linux .
```

Esto descarga la imagen base `python:3.11-slim`, instala las dependencias y copia los archivos de la aplicación.

---

4. Levantar los servicios con `docker compose up`

```
docker compose up
```

Docker Compose levanta el contenedor con la configuración definida en `docker-compose.yml` (variables de entorno, volúmenes y red).

---

5. Acceder a la aplicación

La aplicación es una ventana de escritorio gráfica — **no expone puertos de red ni tiene URL**. Al ejecutar `docker compose up`, la ventana del chatbot aparece directamente en tu escritorio.

Si quieres ejecutarla sin Docker (más rápido para desarrollo):

```
cd src
python app.py
```

---

6. Exposición de puertos

Puerto	Protocolo	Uso
6000	TCP	X11 — protocolo gráfico para mostrar la ventana en el escritorio del host

Esta aplicación no expone un puerto HTTP de navegador. En su lugar utiliza el protocolo **X11**, que es el estándar para mostrar interfaces gráficas de forma remota en sistemas Unix/Linux.

El puerto **6000** es el puerto estándar de X11 y está declarado en el `Dockerfile` con la instrucción `EXPOSE 6000`. Esto indica a Docker que el contenedor usa ese canal para comunicarse con el servidor gráfico del host.

En la práctica:

- En **Windows**, VcXsrv escucha en ese puerto y recibe la ventana del contenedor.
- En **Linux**, el servidor X11 del sistema operativo hace lo mismo a través de `/tmp/.X11-unix`.
- La variable `DISPLAY` del `docker-compose.yml` le dice al contenedor a qué dirección y puerto debe enviar la interfaz gráfica.

🐳 Explicación de los archivos Docker

`Dockerfile`

```
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    python3-tk \
    tk-dev \
    x11-apps \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
```

```
COPY src/app.py .
COPY src/db.json .

ENV DISPLAY=:0

EXPOSE 6000

CMD ["python", "app.py"]
```

Instrucción	Explicación
`FROM python:3.11-slim`	Imagen base ligera con Python 3.11 (menor tamaño)
`RUN apt-get install python3-tk`	Tkinter no viene preinstalado en la imagen slim
`WORKDIR /app`	Define el directorio de trabajo dentro del contenedor
`COPY src/app.py .`	Copia solo los archivos necesarios, no todo el proyecto
`COPY src/db.json .`	Copia la base de conocimiento al contenedor
`ENV DISPLAY=:0`	Pantalla por defecto (el compose la sobreescribe con la IP real)
`EXPOSE 6000`	Documenta que el contenedor usa el puerto 6000 (X11) para la interfaz gráfica
`CMD ["python", "app.py"]`	Comando que se ejecuta al iniciar el contenedor

`docker-compose.yml`

```yaml
services:
  chatbot:
    build: .
    image: chatbot-linux:latest
    container_name: chatbot_linux
    environment:
      - DISPLAY=TU_IP:0.0
    volumes:
      - ./src/db.json:/app/db.json
    restart: "no"
```

Sección	Explicación
`build: .`	Construye la imagen a partir del Dockerfile local
`image`	Nombre que tendrá la imagen construida
`environment: DISPLAY`	Redirige la ventana gráfica al escritorio del host

Sección	Explicación
`volumes`	Monta `db.json` como volumen persistente: puedes añadir comandos sin reconstruir la imagen
`restart: "no"`	El contenedor no se reinicia solo al cerrarse la ventana

🔧 Posibles problemas y soluciones

La ventana no aparece / `cannot connect to X server`

Sistema	Solución
Windows	Abre VcXsrv (XLaunch) con "Disable access control" activado y verifica que la IP en `DISPLAY` es correcta
Linux	Ejecuta `xhost +local:docker` antes de `docker compose up`

`The "DISPLAY" variable is not set`

El `docker-compose.yml` tiene `${DISPLAY}` en vez de una IP fija. En Windows esta variable no existe. Sustitúyela por tu IP directamente:

```
- DISPLAY=192.168.1.50:0.0
```

`version is obsolete` al hacer `docker compose up`

Es un aviso cosmético. Elimina la línea `version: "3.8"` del `docker-compose.yml` para que desaparezca.

`db.json not found` al arrancar el contenedor

Verifica que el archivo está en `src/db.json` y que el volumen en `docker-compose.yml` apunta a:

```
- ./src/db.json:/app/db.json
```

Error al montar volumen: `not a directory`

Ocurre si el nombre del archivo no coincide en ambos lados del volumen, o si Docker interpreta la ruta como carpeta. Comprueba que `src/db.json` existe como archivo (no como carpeta).

Docker Desktop no responde

Abre Docker Desktop desde el menú inicio y espera a que el icono de la ballena en la barra de tareas quede fijo (sin animación). Tarda entre 30 y 60 segundos.

✨ Contribuciones y organización del proyecto

Flujo de trabajo con ramas

```
main                    ← rama principal, versión estable
  ├── set-docker        ← configuración del Dockerfile y docker-compose
  └── feat-smart-search ← búsqueda difusa con difflib
```

Cada funcionalidad se desarrolla en una rama propia. Cuando está lista, se fusiona en `main` mediante un **Pull Request** con revisión antes de aceptar.

Pasos para contribuir

```
# 1. Crear una rama nueva desde main
git checkout -b feat/nombre-de-la-funcionalidad

# 2. Desarrollar y hacer commits descriptivos
git add .
git commit -m "feat: descripción clara del cambio"

# 3. Subir la rama
git push origin feat/nombre-de-la-funcionalidad

# 4. Abrir un Pull Request en GitHub hacia main
```

Convención de commits

Prefijo	Uso
`feat:`	Nueva funcionalidad
`fix:`	Corrección de un error
`docs:`	Cambios solo en documentación
`chore:`	Tareas de mantenimiento (dependencias, config)

Ampliar la base de conocimiento

Edita `src/db.json` y añade entradas con este formato:

```
{
  "pregunta": "nano",
  "respuesta": "El comando 'nano' abre un editor de texto en la terminal. Ejemplo:
'nano archivo.txt'.",
```

```
  "url": "https://man7.org/linux/man-pages/man1/nano.1.html"
}
```

Si el contenedor está corriendo con el volumen montado, el cambio se aplica sin necesidad de reconstruir la imagen.

🖼️ Diagrama de arquitectura

```
┌──────────────────────────────────────────┐
│              Host (Windows/Linux)        │
│                                          │
│  ┌──────────────┐      ┌──────────────────┐
│  │  VcXsrv /    │◄─────│  Docker Desktop  │
│  │  X11 server  │      │                  │
│  └──────────────┘      │  ┌──────────────┐
│        ▲               │  │ Contenedor   │
│        │ ventana       │  │              │
│        │ gráfica       │  │  app.py      │
│  ┌──────────────┐      │  │  db.json     │
│  │   Usuario    │      │  └──────────────┘
│  └──────────────┘      └──────────────────┘
│                                          │
│  Volumen: ./src/db.json ──► /app/db.json │
└──────────────────────────────────────────┘
```